

Python #55

Доволі часто під час розробки додатків з використанням баз даних може знадобитися додати видалити або модифікувати базу даних. Одним із варіантів модифікація бази даних є додавання, видалення або зміна назви колонки таблиці.

Для додавання нового стовпчика в таблицю використовується наступний синтаксис:

```
ALTER TABLE table_name  
ADD new_column_name column_definition;
```

Де :

- *table_name* - ім'я таблиці, яку потрібно змінити.
- *new_column_name* - Назва нового стовпця, який потрібно додати до таблиці.
- *column_definition* - Тип даних і визначення стовпця (NULL або NOT NULL тощо)

Як приклад додавання стовбцю **age** в таблицю **users**.

```
ALTER TABLE users  
ADD age text;
```

Таким чином мною було додано ще 3 рядка до моєї таблиці тому результуюча таблиця має наступний вигляд:

	id	name	email	password	astName ▼ ¹	age	tell
	Філ...	Фільтр	Фільтр	Фільтр	Фільтр	Фільтр	Фільтр
1	2	Volodymyr	Skool@gmail.com	pbkdf2:sha2...	NULL	NULL	NULL

Дані команди є командами SQLite і запускаються таким же чином як і команди на створення таблиць, але при повторному запуску даної команди відбудеться помилка.

Наступним етапом є відображення отриманих даних для цього ми створимо сторінку **profile** яка буде відображатися після логіну тобто замінимо в шляху в функції перенаправлення (створювалося на попередніх заняттях).

Функція виклику сторінки матиме наступний вигляд:

```
@app.route("/profile")
@login_required
def profile():
    return render_template('profile.html', name=current_user)
```

Де **current_user** є імпортованою функцією з **flask_login**. Цей об'єкт **current_user** представляє собою базу даних, і ми можемо отримати доступ до всіх атрибутів цього користувача, використовуючи точну нотацію. Тобто в нашому випадку він отримує доступ до всіх функцій **UserLogin**, в якому в свою чергу необхідно додати гетери для усіх полів.

```
class UserLogin:
    def fromDB(self, user_id, db) :
        self.__user = db.getUser(user_id)
        return self

    def create(self, user):
        self.__user = user
        return self

    def is_authenticated(self):
        return True

    def is_active(self):
        return True

    def is_anonymous(self):
        return False

    def get_id(self) :
        return str(self.__user['id'])

    def get_name(self) :
        return str(self.__user['name'])

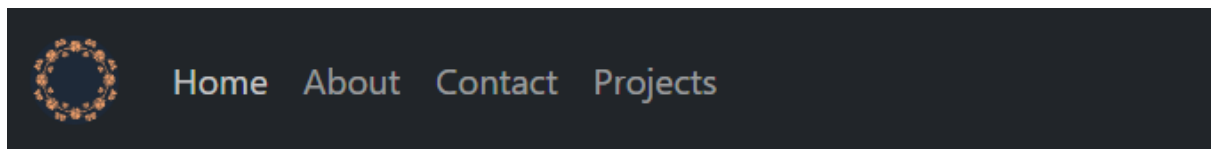
    def get_lastName(self) :
        return str(self.__user['lastName'])

    def get_age(self) :
        return str(self.__user['age'])
```

Після чого можна перейти до створення html:

```
{% extends "layout.html" %} {% block content %}|  
  
<p><b>Name :</b> {{ name.get_name() }}</p>  
<p><b>Last Name :</b> {{ name.get_lastName() }}</p>  
<p><b>Age :</b> {{ name.get_age() }}</p>  
<p><b>eMail :</b> {{ name.get_email() }}</p>  
<p><b>Tell :</b> {{ name.get_tell() }}</p>  
  
{% endblock %}
```

Після входу користувача вас повинно пере направити приблизно на наступну сторінку:



Name : Volodymyr

Last Name : None

Age : None

eMail : Skool@gmail.com

Tell : None

Завдання 1.

Додати даний функціонал до розробленого вами сайту на передніх заняттях. Додати також до бази даних стовбці які відповідають за телефон і прізвище користувача який увійшов.

Завдання 2.

Зберегти зміни на git.